
University of Kansas

**Group 18 Minesweeper
Software Architecture Document**

Group 18 Minesweeper	
Software Architecture Document	Date: 20/Sep/26
EECS581-Minesweeper-SAD	

Software Architecture Document

1. Introduction

The Minesweeper project will be a single-player puzzle game that will be played on a 10x10 grid. The player will be able to uncover the cells on the board while still attempting to avoid any randomly placed mines. When a mine-free cell is uncovered, it displays a number from 0-8 that will indicate the number of mines that are adjacent to that cell. If there are zero adjacent mines surrounding then the cells will recursively uncover.

Once the game starts the player will have to select a number of mines and they will be ranging from 10-20. The mines will be placed randomly across the board, and the first cell selected by the player will be guaranteed to be mine free. There is also a feature where the player can place and remove flags on the covered cells that the player suspects to be a mine in those locations. The game can end in two ways. First the player can uncover all the cells without a mine and result in a victory. However, if the player picks a cell that has a mine underneath the player's result will result in a Loss.

1.1 Purpose

The purpose of this Software Architecture document is to help describe how the Minesweeper system is designed and organized. It will help provide an overview of the major components of the system, and how the information moves throughout the entire system. This document is intended to help the current development team and the next development team for Project 2 to help understand the structure of the Minesweeper system. This will make it easier for the system to be maintained, modified, and understood where the previous development group left off. This document will cover the system architecture, components, data flow, key data structures, interfaces, constraints, and assumptions.

2. System Architecture Overview

2.1 High-Level Architecture

The Minesweeper system uses a modular, component-based architecture that will have four major components: The Board Manager, Game Logic, Input Handler, and User Interface. Each of these components will have assigned its own responsibilities, however they will be communicating with one another when specific conditions are met. Separating the system into these four components will help organize this program into more understanding, and easier components to maintain.

Board Manager: The Board Manager, implemented in `sprites.py`, will have the responsibility for managing the 10x10 Minesweeper board. It will help keep track of the state of each cell, including whether a cell is uncovered, covered, flagged, or does contain a mine.

Game Logic: The Game Logic, implemented in `main.py`, will be responsible for handling the rules and behavior of the Minesweeper game. This will include the mine placement, the uncovering of cells, recursively uncovering surrounding cells, determining the number of adjacent mines, and determining whether or not the player has won or lost.

Input Handler: The Input Handler, implemented in `main.py`, will be responsible for processing actions made by the player. Features may include selecting a cell to uncover or placing and removing a flag. Then the Input Handler will send the player's actions to the Game Logic.

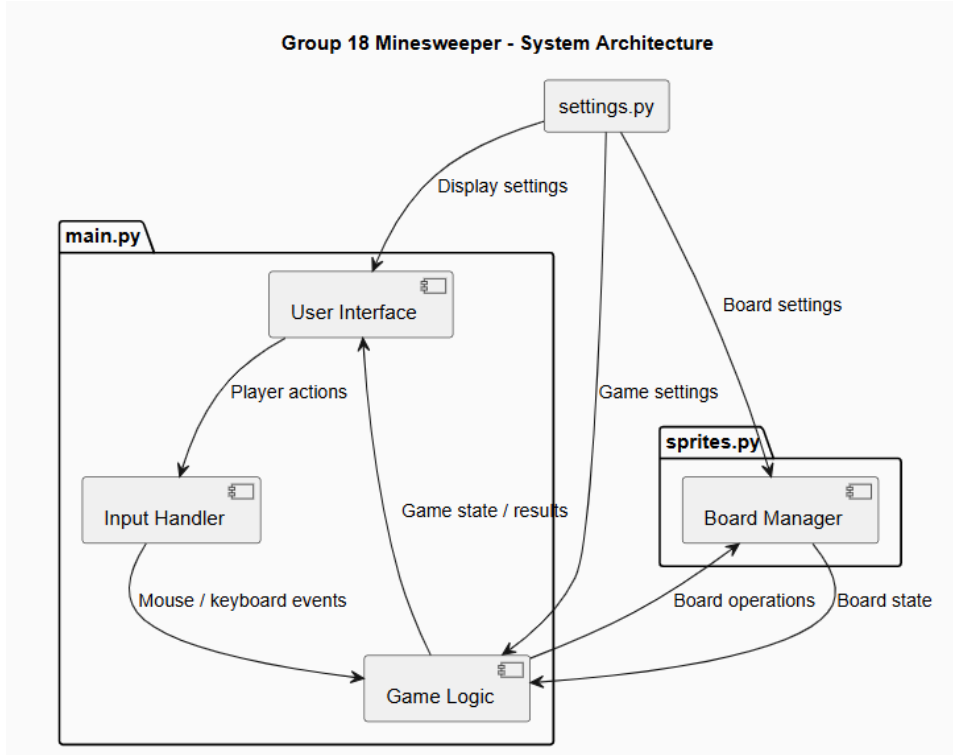
User Interface: The User Interface, implemented in `main.py`, will be responsible for having the display of the game to the player. The display will be a 10x10 board with uncovered cells, covered cells, flags, mine information, and the current status of the game.

At a high level, these four components will all be in unison to control the Minesweeper game. The Input Handler will first receive the actions made by the Player, next the Game Logic will determine the actions

Group 18 Minesweeper	
Software Architecture Document	Date: 20/Sep/26
EECS581-Minesweeper-SAD	

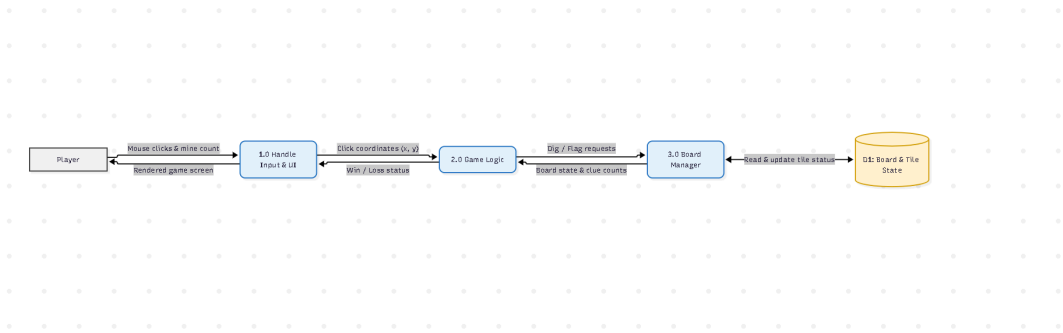
the player makes that can affect the game, next the Board Manager will maintain the current state of the board, and finally the User Interface will display the updated game information to the player.

2.2 Architecture Diagram



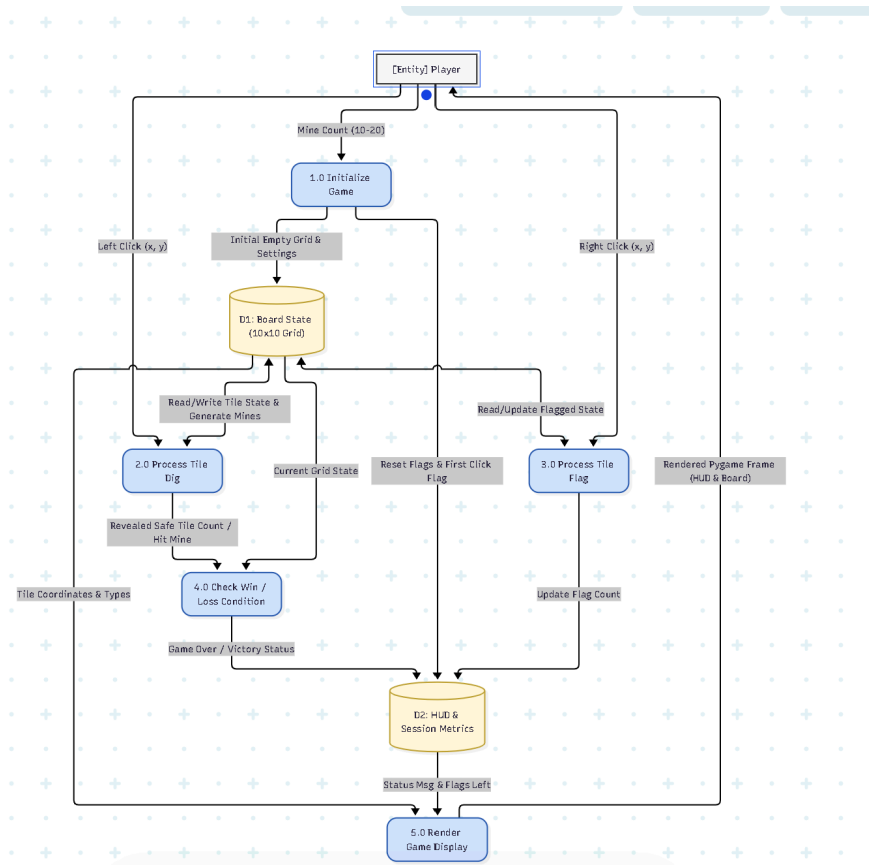
2.3 Data Flow

High level:



Group 18 Minesweeper	
Software Architecture Document	Date: 20/Sep/26
EECS581-Minesweeper-SAD	

Detailed:



3. Architectural Components

Board Manager

The Board Manager maintains the game board and the state of every cell. It is implemented through the Board and Tile classes in `sprites.py`.

- Creates and stores the 10x10 board.
- Stores each cell's position, type, clue number, revealed state, and flagged state.
- Tracks whether cells are covered, uncovered, flagged, empty, numbered, or mines.
- Provides the current board state to the Game Logic and User Interface.

Game Logic

The Game Logic controls the rules and progression of the game.

- Randomly places mines after the first click while keeping the first-click area safe.
- Calculates the number of adjacent mines for each non-mine cell.
- Uncovers selected cells and recursively reveals connected empty cells.
- Prevents flagged cells from being uncovered.

Group 18 Minesweeper	
Software Architecture Document	Date: 20/Sep/26
EECS581-Minesweeper-SAD	

- Detects a loss when a mine is uncovered.
- Detects victory when every non-mine cell has been revealed.

Input Handler

The Input Handler receives player actions and converts them into game commands.

- Accepts and validates the selected mine count.
- Detects left-clicks for uncovering cells.
- Detects right-clicks for placing or removing flags.
- Converts mouse coordinates into board coordinates.
- Handles window closing and post-game restart input.

User Interface

The User Interface displays the current game state.

- Draws the 10x10 grid and its row and column labels.
- Displays covered, uncovered, flagged, numbered, and mined cells.
- Displays the remaining flag count.
- Displays Playing, Victory, or Game Over.
- Refreshes the screen after each player action.

4. Key Data Structures

2D Matrix - The Board State

- **Variable:** `self.board_list`
- Represents the primary data structure of the game. It models the 10×10 grid as a column-major 2D array, enabling instant O(1) spatial lookups (`board_list[x][y]`) when converting mouse pixel clicks into board cells and inspecting adjacent neighbors.

Hash Set - The Visited Tracker

- **Variable:** `self.dug`
- Crucial for the recursive flood-fill algorithm in `Board.dig()`. Storing visited (x, y) coordinate tuples in a hash set guarantees O(1) membership lookup times, preventing infinite recursion loops and redundant processing when clearing blank areas.

Hash Set - The First-Click Exclusion Zone

- **Variable:** `safe_zone`
- Stores the 3×3 grid of (x, y) coordinate tuples surrounding the player's opening click. Provides O(1) containment checks during board setup to guarantee that neither the clicked tile nor its immediate neighbors are seeded with mines.

List - Candidate Mine Pool

- **Variable:** `available`
- A list of valid (x, y) coordinate tuples representing every cell outside the `safe_zone`. Serves as the direct

Group 18 Minesweeper	
Software Architecture Document	Date: 20/Sep/26
EECS581-Minesweeper-SAD	

input pool for `random.sample()` to select unique, non-overlapping mine locations.

Hash Map - The Clue Style Table

- **Variable:** NUM_COLORS
- Provides an O(1) lookup table mapping clue numbers (1–8) directly to their corresponding RGB color tuples, avoiding long chains of if/elif branching during every frame render.

5. Interface Description

5.1 User Input

The Input Handler receives player actions and converts them into valid game commands:

- Before play begins, the terminal requests an integer from 10 to 20 for the mine count and repeats the prompt when the entry is invalid.
- A left-click on an unflagged board cell uncovers it. On the first valid click, mines and number clues are created while the selected cell and its neighbors remain safe.
- A right-click on a covered cell places or removes a flag and updates the flag count.
- A click after Victory or Game Over starts a new round, while closing the window exits the game.

5.2 User Interface

The User Interface displays the current board and game state to the player:

- A 10x10 grid is labeled with columns A-J and rows 1-10.
- Cells are shown as covered, uncovered, or flagged. Uncovered mines are displayed after a loss.
- Uncovered clue cells display the number of adjacent mines using number-specific colors, while empty cells display no number.
- The status area shows the remaining flags and the current state: Playing, Victory, or Game Over.
- The display refreshes after each action so the board, flag count, and status remain current.

5.3 Component Interfaces

The three source files communicate through shared objects, method calls, and configuration values:

- `main.py` receives mouse input, converts screen coordinates into board coordinates, and sends the requested action to the Board object.
- `sprites.py` provides the Board and Tile classes, which store the 10x10 grid and each cell's type, clue number, revealed state, and flagged state.
- Game Logic calls `Board.place_mines()`, `Board.place_clues()`, and `Board.dig()` to place mines, calculate clues, uncover cells, and recursively reveal empty areas.
- The Game class reads the updated board state to detect victory or loss, while `Board.draw()` renders that state through the User Interface.
- `settings.py` supplies the shared board dimensions, window measurements, frame rate, and display colors used by `main.py` and `sprites.py`.

Group 18 Minesweeper	
Software Architecture Document	Date: 20/Sep/26
EECS581-Minesweeper-SAD	

6. Architectural Constraints and Assumptions

6.1 Constraints

The Minesweeper system must follow the architectural constraints listed below:

- The game must use a fixed 10x10 grid with columns labeled A-J and rows numbered 1-10.
- The number of mines must be user-specified and limited to 10-20 mines.
- Mines must be randomly placed when the game begins.
- The first clicked cell must be mine-free.
- All cells must initially be covered and contain no flags.
- The system must allow for uncovering cells, flagging/unflagging cells, and detecting game victory or loss.
- The Minesweeper game must allow for an easy-to-use interface that shows the game board, remaining mine/flag count, cell states (covered, uncovered, flagged, mined), and game status.

6.2 Assumptions

The Minesweeper game is based on the following assumptions:

- The game is single-player.
- The board size remains fixed at 10x10 throughout the game.
- The user provides a valid mine count between 10 and 20 before the game begins.
- User input follows the expected game actions throughout the game.